

Indian Institute of Information Technology Bhagalpur

(An Institute of National Importance Under Act of Parliament)



Computer Graphics - CS306

Assignment - 01

Submitted by:

Priyanshu (2201124CS)

Class: B. Tech. – III

Semester: VI

Course Instructor: **Dr. Ujjwal Biswas**

Junior Technical Superintendent: **Mr. Luv Pandey**

Department of Computer Science & Engineering

IIIT BHAGALPUR, BIHAR 813210, INDIA

Jan-May 2025

Contents

1. Introduction to Graphics Systems

- 1.1 What is Visualization?**
- 1.2 Graphical User Interface (GUI)**
- 1.3 Video Display Devices**
- 1.4 Raster Scan Systems**
- 1.5 Random Scan Systems**
- 1.6 Vector Devices**
- 1.7 Cathode Ray Tube Terminals**
- 1.8 Input Devices**

2. Display and Drawing of Graphics Primitives

- 2.1 Display of Colors- Look Up Tables**
- 2.2 Display of Gray Shades**
- 2.3 Half Toning**
- 2.4 Point**
- 2.5 Line**
- 2.6 Polygon**
- 2.7 Circle**
- 2.8 Curves and Text**

3. Coordinate Conventions

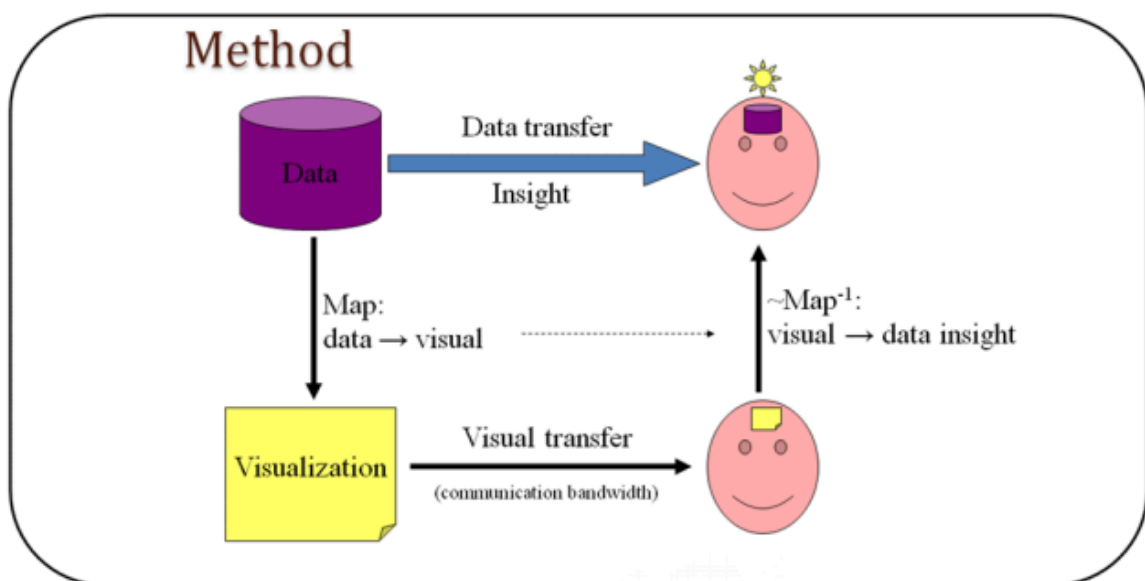
- 3.1 Scan Conversion a Line**
- 3.2 Scan Conversion a Circle**
- 3.3 Scan Conversion an Ellipse**
- 3.4 World Coordinates**
- 3.5 Device Coordinates**
- 3.6 Normalized Device Coordinates**
- 3.7 What is Coordinate Reference Frames (CRFs)**
- 3.8 Zooming by Changing CRFs**
- 3.9 Panning by Changing CRFs**

1 Introduction to Graphics Systems

1.1 What is Visualization?

The use of computer-supported, interactive, visual representations of data to amplify cognition. Transformation of data or information into pictures can be done using visualization techniques. It is a tool for learning and understanding.

The Computer Graphics and Visualization group consists of researchers in image processing, computer graphics, visualization, visual analytics, GPU programming, simulation, and geometry processing. The primary goals of this group are the analysis, synthesis, understanding, and manipulation of visual data such as images, video sequences, and 3D geometric content



Reference:

<https://ebooks.inflibnet.ac.in/csp06/chapter/visualization-definition-and-history-of-visualization/>

1.2 What is Graphical User Interface (GUI)?

The common and useful component of electronics such as computers, tablets, and smartphones is a visual interface called a Graphical User Interface (GUI). The graphical user interface (GUI) shows useful items that indicate actions the user can perform and transmit information as per

requirement. When interacting with the objects, the user can modify or simplify their size, color as well as visibility. As the text command-line interfaces in the system were complex and challenging to master, GUIs were developed. It is currently the accepted practice in software applications or system programming for required user-centered design.

A system of interactive visual components for a computer or system software is called a GUI (graphical user interface). GUI is the interface that uses graphical elements to let people interact as per requirement with electronic devices including computers, laptops, tablets, and smartphones. In terms of human-computer interaction systems or technology, it's a very important component of software application programming since it substitutes actions for the text-based commands in the system. Whether it's a text file, object, image, or video as per requirement, it displays all types of required content a user could envision in the system. Additionally, it can be featured in the gaming platform where the resolution is visible or optimal.

History of GUI:

In the year of 1981, Alan Kay, Douglas Engelbart, and other researchers at Xerox PARC (Palo Alto Research Centre) developed the GUI technology. A GUI-equipped Lisa computer was later released by Apple as per requirement on January 19, 1983. Before there was a graphical user interface, communication was done via the command-line interface, or CLI to manage the overall system. The end users did not find the CLI particularly enjoyable to use and modify, so they were unfamiliar with all of the user-beneficial commands. Thus, the interface was put into place to fill this required gap. One of the most important techniques or features, as the GUI highlights, is "ease of use." The individual system of Xerox 8010 Information System served as the first GUI-centric machine operating model in the technology.

Components of GUI:

Pointers: The pointer appears on the user's screen as a marking symbol. The pointer moves on to choose instructions and objects as per requirement.

Icons: Icons allude to tiny visual representations of windows, documents, actions, and other things on the display screen to simplify. A pointer and pointing device can be used by the user to carry out the initial tasks for the overall processes.

Pointing tool: At the initial stages, the pointing tool enables the user to select and move the required pointer items on the screen, including a trackball or mouse. It is the most beneficial tool in GUI.

Desktop: The desktop is the screen that is contained within the icons and user beneficial.

Features of GUI

- > The graphical user interface (GUI) is very easy to use and the user can modify and simplify the requirements.
- > The required software, documents, or a few relevant programs are reflected in the icons on the user interface to control the overall processes properly.
- > A graphical user interface (GUI) has several features as per requirement, such as tabs, a menu, pointers, and various other types of things to simplify and process smoothly.

Advantages of GUI

- > A graphical user interface (GUI) is basically seen as more intuitive for users than a text-based command-line interface as per requirement, such as MS-DOS or the Unix-like operating system shell process.
- > It is incredibly user-friendly and readily available to all and for novices, the user interface is rather easy to understand and uncomplicated as per requirement.
- > GUI represents the now-hidden lines of command with the required graphic elements.
- > The end users must commit required instructions to memory for the software to function properly.

Disadvantages of GUI

- > An interface which is very much difficult to use will make the required tasks more difficult for the user to complete and less effective overall processes.
- > Gamers are still much affected negatively by a poor interface or surface system, therefore it remains a problem for them to control the overall system.

Conclusion

GUI enables the users to work with and display many required programs simultaneously to process. Despite the rapid advancements in technology

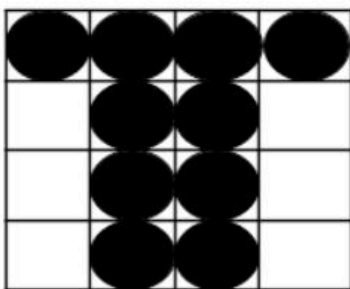
and science, the Graphical User Interface continues to be a vital component of users online communications. Users can minimize the required strokes by using shortcut keys provided by the GUI. By using just two keys to do multiple activities, a user can save time and boost productivity as per the system. All it takes to obtain properly a function is a single click. GUI has taken over as the required primary interface for computers and mobile devices or systems because it is very much simple to use and comprehend.

Reference:

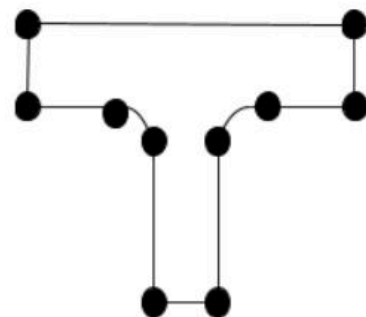
<https://www.geeksforgeeks.org/what-is-graphical-user-interface/>

1.3 Display Processor in Computer Graphics:

Display Processor is the interpreter or a hardware that converts display processor code into picture. The Display Processor converts the digital information from CPU to analog values. The main purpose of the Digital Processor is to free the CPU from most of the graphic chores. The Display Processor digitize a picture definitions given in an application program into a set of pixel intensity values for storage in the frame buffer. This digitization process is called Scan Conversion.



A Character (T)
defined in a grid of
pixel positions.



A Character (T)
defined as a curve
outline.

Silent features of Display Processor:

- > Display processors includes functions such as generating various line styles, displaying color area and performing transformations and manipulations on display object.
- > Display Processor was used before the GPU (Graphics Display Processor). Video Controller is the most widely used Display device that is based on CRT (Cathode Ray Tube).
- > In addition with the system memory, Display Processor have a separate memory area.

Display Devices:

There are large number of Display Devices available to free the CPU from graphic chores:

- > Refresh Cathode Ray Tube
- > Random Scan and Raster Scan
- > Color CRT Monitors
- > Direct View Storage Tubes
- > Flat Panel Display
- > Lookup Table

Reference:

<https://geeksforgeeks.org/display-processor-in-computer-graphics/>

1.4 Raster Scan Systems:

Raster scan, or raster scanning, is the rectangular pattern of image capture and reconstruction in television. By analogy, the term is used for raster graphics, the pattern of image storage and transmission used in most computer bitmap image systems.

Several processing units: Contains a special purpose processor, called video controller or display controller.

Video controller: A fixed area of the system memory is reserved for the frame buffer, and the video controller is given access to the framebuffer memory.

- > Two registers are used to store the coordinates of the screen pixels.
- > The value is stored in the frame buffer for this pixel position is then retrieved and used to set the intensity of the CRT beam. Initially the x register is set to 0 and the y register is set to Ymax.
- > Then the x register is incremented by 1, and the process repeated for the next pixel on the top scan line.

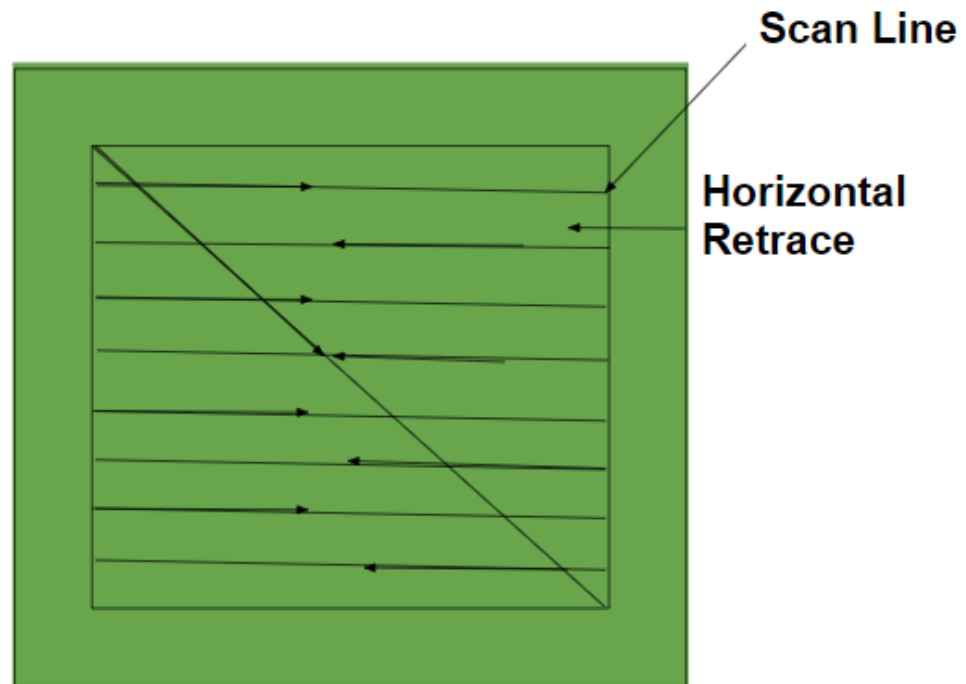
Computer Graphics is the manipulation of the image in a pictorial manner. The rendering can be done in this manipulation and creation of the new image and hence computer graphics is used. Even a raster scan is an important aspect when it comes to computer graphics.

Raster Scan Displays are the most common type of graphics monitor which employs CRT. It is based on television technology.

Working of Raster Scan Display

- > In the Raster scan system, the electron beam is used as a fixed path on the screen. The electron beam starts from the top left corner of the screen and follows the horizontal path until it reaches the end of the line. This defines a scan line. During the scan, the intensity of the beam is modulated according to the pattern of the desired image along the line.
- > After reaching the right corner of the screen, the beam is switched off and it is moved back to the left edge/point of the new row as the new starting point. This defines horizontal retrace.
- > Picture definition is positioned in a memory area which is technically known as Frame Buffer. This memory area holds up the set of intense values for all the screen points. Stored intensity values are then retrieved from the frame buffer and used to display images on the screen.
- > Now the scanning is continued until it reaches the bottom right corner of the screen. After it reaches the bottom right corner of the screen it defines one scan is completed. Each screen point is known as a pixel.
- > After one scan is completed the electron beam is re-positioned at the top left corner of the screen for the next scan. The repositioning process of the electron beam bottom right corner to top left corner is referred to as vertical retrace.

The diagram is represented as follows:



Advantages

- > Cost for raster scan devices is cheaper/low.
- > Doesn't require an intelligent electron beam.
- > Used for animation purposes.

Disadvantages

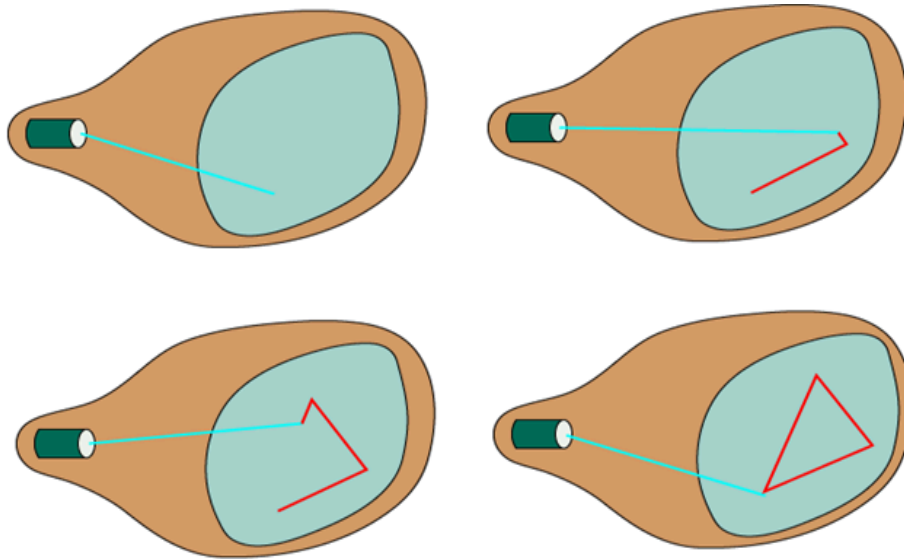
- > Low resolution.
- > Requires more memory.
- > Electron beam coordinated to the whole screen not exclusively to those parts of the screen where the picture is to be drawn.

Reference:

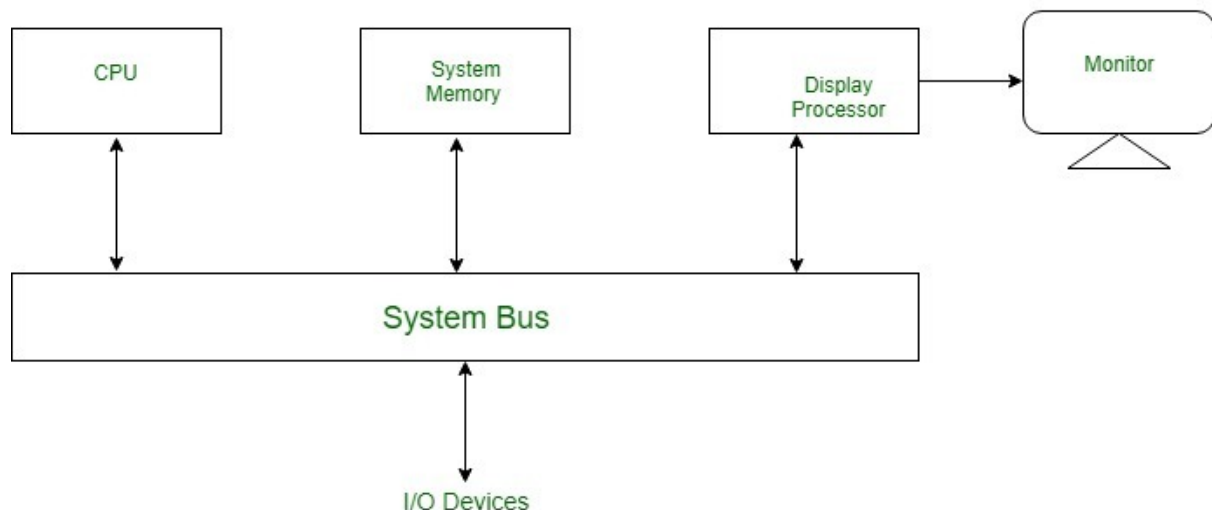
<https://discuss.boardinfinity.com/t/what-is-raster-scan-systems/11492#advantages-2>

1.5 Random Scan Systems

Random Scan System uses an electron beam which operates like a pencil to create a line image on the CRT screen. The picture is constructed out of a sequence of straight-line segments. Each line segment is drawn on the screen by directing the beam to move from one point on the screen to the next, where its x & y coordinates define each point. After drawing the picture. The system cycles back to the first line and design all the lines of the image 30 to 60 time each second. The process is shown in fig:



Random-Scan Display Processors: Input in the form of an application program is stored in the system memory along with graphics package. Graphics package translates the graphic commands in application program into a display file stored in system memory. This display file is then accessed by the display processor to refresh the screen. The display processor cycles through each command in the display file program. Sometimes the display processor in a random-scan is referred as **Display Processing Unit / Graphics Controller**. The structure of a simple random scan is shown below:



ADVANTAGES:

- > Higher resolution as compared to raster scan display.
- > Produces smooth line drawing.
- > Less Memory required.

DISADVANTAGES:

- > Realistic images with different shades cannot be drawn.
- > Colour limitations.

1.6 Vector Devices:

Vector devices are tools or technologies that use vector-based representation for various applications in graphics, mathematics, engineering, and computing. Here's a breakdown of different contexts in which vector devices are used:

1. Graphics and Display Systems

In computer graphics, vector devices are used to render images using geometric primitives like points, lines, and curves rather than pixels.

Examples include:

- **Vector Monitors:** Display devices that directly draw lines between points rather than refreshing pixels, historically used in early arcade games and oscilloscopes.
- **Plotters:** Machines that draw vector graphics on paper using pens, commonly used for engineering and architectural drawings.

Advantages:

- Smooth and scalable graphics without pixelation.
- Efficient for line drawings and simple geometric designs.

Disadvantages:

- Less suited for detailed images like photographs.

2. Mathematical and Engineering Applications

Vector devices are used for calculations and representations involving vectors:

- **Vector Analyzers:** Used in electrical engineering for testing networks with varying frequencies.
- **Oscilloscopes with Vector Modes:** Visualize signals in a 2D space.

3. CAD (Computer-Aided Design) Systems

Vector-based devices and software are fundamental in CAD systems for drafting and modeling. Examples:

- **AutoCAD:** Uses vector graphics to create precise 2D and 3D designs.

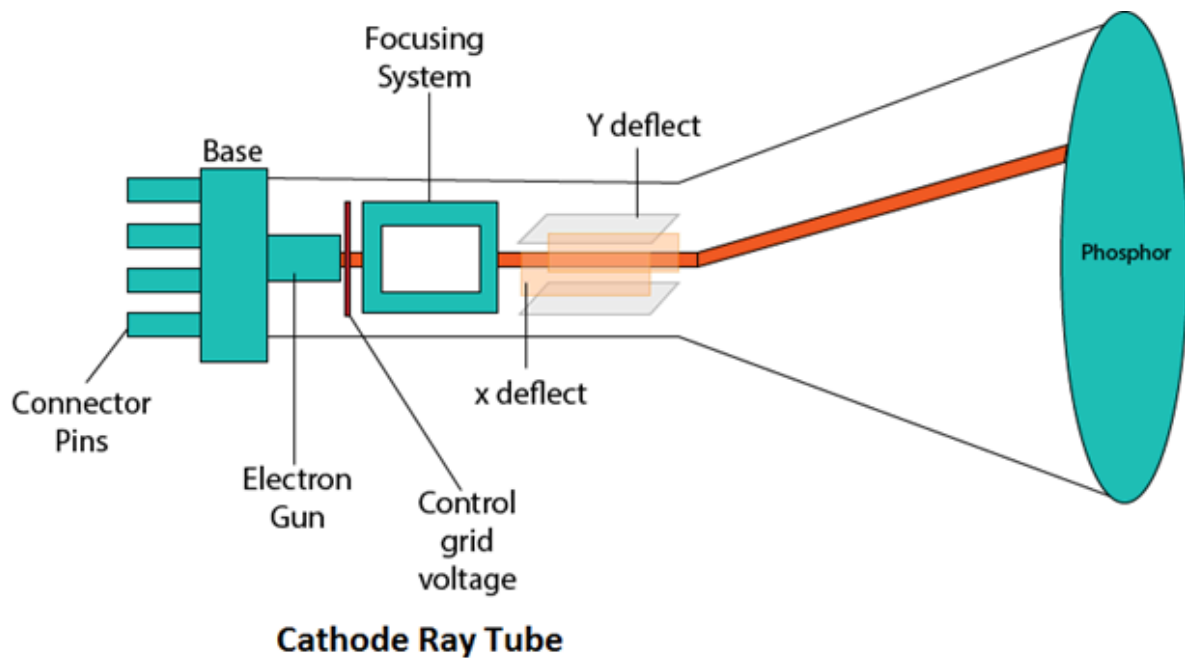
4. Gaming and Simulation

Early arcade games like Asteroids (1979) used vector graphics, providing sharp and smooth line-based visuals.

1.7 Cathode Ray Tube Terminals:

CRT stands for Cathode Ray Tube. CRT is a technology used in traditional computer monitors and televisions. The image on CRT display is created by firing electrons from the back of the tube of phosphorus located towards the front of the screen.

Once the electron heats the phosphorus, they light up, and they are projected on a screen. The color you view on the screen is produced by a blend of red, blue and green light.



Components of CRT:

Main Components of CRT are:

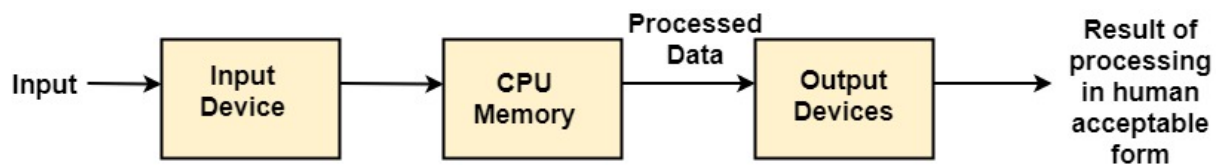
- 1. Electron Gun:** Electron gun consisting of a series of elements, primarily a heating filament (heater) and a cathode. The electron gun creates a source of electrons which are focused into a narrow beam directed at the face of the CRT.
- 2. Control Electrode:** It is used to turn the electron beam on and off.
- 3. Focusing system:** It is used to create a clear picture by focusing the electrons into a narrow beam.
- 4. Deflection Yoke:** It is used to control the direction of the electron beam. It creates an electric or magnetic field which will bend the electron beam as it passes through the area. In a conventional CRT, the yoke is linked to a sweep or scan generator. The deflection yoke which is connected to the sweep generator creates a fluctuating electric or magnetic potential.
- 5. Phosphorus-coated screen:** The inside front surface of every CRT is coated with phosphors. Phosphors glow when a high-energy electron beam hits them. Phosphorescence is the term used to characterize the light given off by a phosphor after it has been exposed to an electron beam.

Reference:

<https://www.javatpoint.com/computer-graphics-cathode-ray-tube>

1.8 Input Devices:

The Input Devices are the hardware that is used to transfer transfers input to the computer. The data can be in the form of text, graphics, sound, and text. Output device display data from the memory of the computer. Output can be text, numeric data, line, polygon, and other objects.

**These Devices include:**

1. Keyboard
2. Mouse
3. Trackball
4. Spaceball
5. Joystick
6. Light Pen
7. Digitizer
8. Touch Panels
9. Voice Recognition

Reference: <https://www.javatpoint.com/computer-graphics-input-devices>

2. Display and Drawing of Graphics Primitives

2.1 Display of Colors - Look-Up Tables (LUTs)

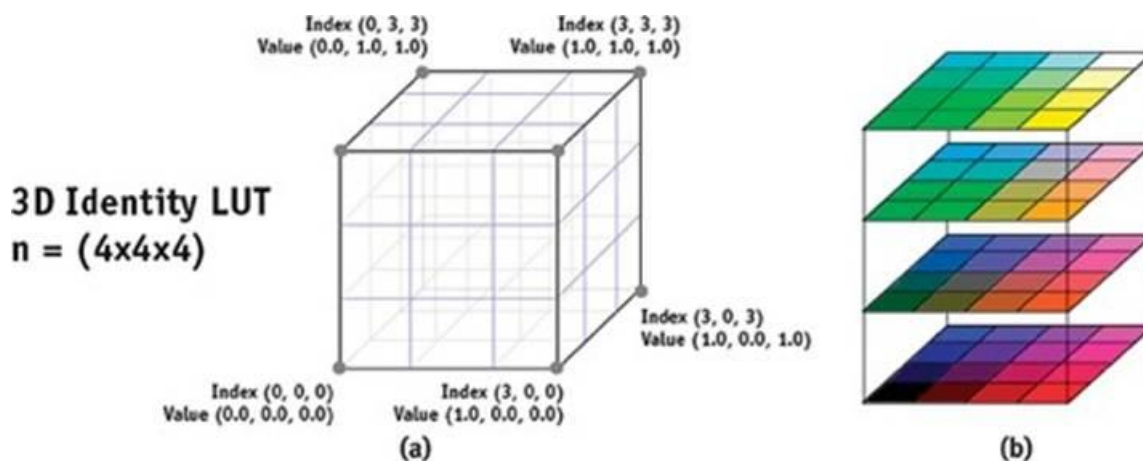
In computer graphics, Look-Up Tables (LUTs) are commonly used for color mapping and color management. A LUT table maps input values (like pixel values) to specific output values, often corresponding to color values. This technique is widely used in

systems with limited color depth or when applying color correction/transformation.

For example:

- In an 8-bit color system, a LUT could map pixel values (0 to 255) to a set of RGB values. The LUT would store the appropriate color for each pixel index, allowing for efficient color rendering.
- In practical terms, LUTs help create effects such as color remapping, contrast enhancement, or gamma correction by adjusting how pixel intensities are mapped to colors.

The idea is that instead of calculating a color dynamically for each pixel, the color is retrieved directly from the LUT, which speeds up the process.

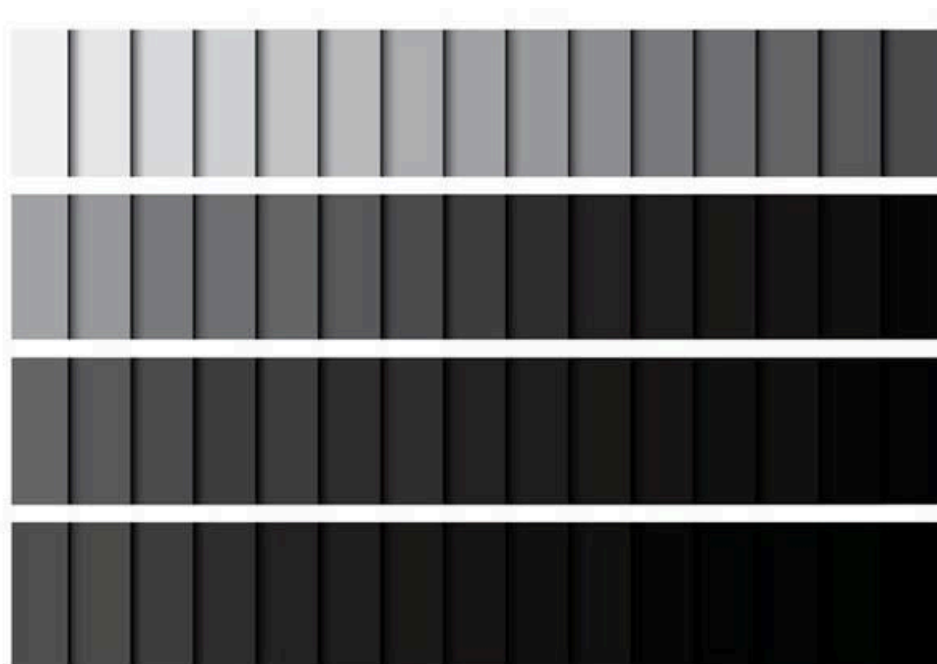


2.2 Display of Gray Shades

Grayscale (or grayscale) refers to the use of shades of gray, which are produced by varying the intensity of light without using color.

Grayscale is commonly used in images where color information is not needed, or in situations with limited display capabilities (like printers or early monitors).

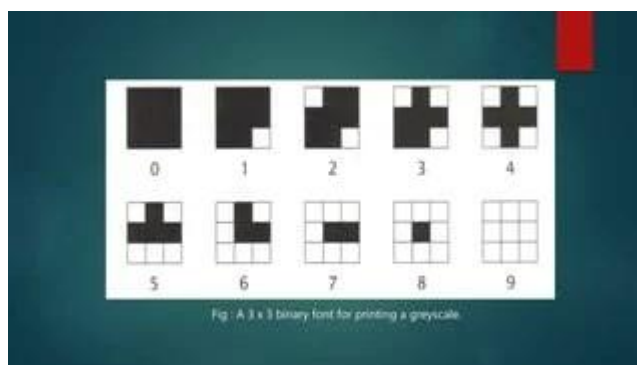
- Gray levels are typically represented using an 8-bit depth, where 0 represents black, 255 represents white, and values in between represent varying shades of gray. The transition from black to white in an image is depicted using these levels.
- The authors of the book may discuss how to achieve smooth shading of gray by interpolating between these discrete levels and ensuring the gradation appears continuous to the human eye.



2.3 Half-Toning

Half-toning is a technique used to simulate continuous-tone images (like photographs) using a limited number of colors or shades. This is typically used in printing, where printing processes are restricted to only black ink or a small number of colors, but the visual impression of continuous tones is still needed.

- Dot patterns are used in half-toning to represent different levels of gray or color. For instance, a large number of small dots closely spaced together can simulate a darker tone, while fewer dots or larger spacing can simulate a lighter tone.
- In *Computer Graphics: Principles and Practice*, the book would cover how these dot patterns are arranged to create visual effects like dithering or error diffusion (where pixel values are adjusted to approximate a smoother tone).

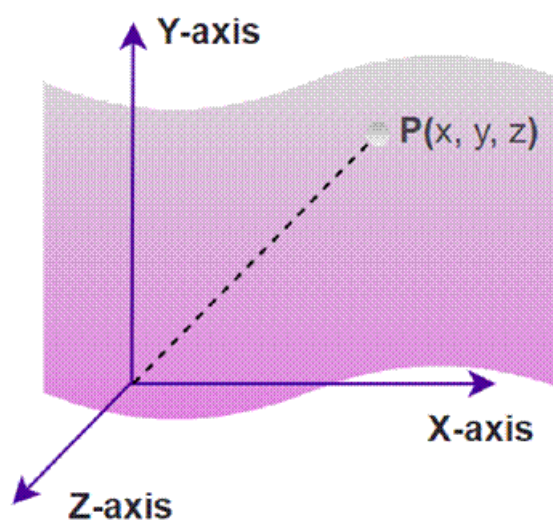




2.4 Point

A point is a basic graphical primitive, often represented by a pair of coordinates (x, y) in 2D space or (x, y, z) in 3D space. A point does not have any dimension; it simply represents a location in space.

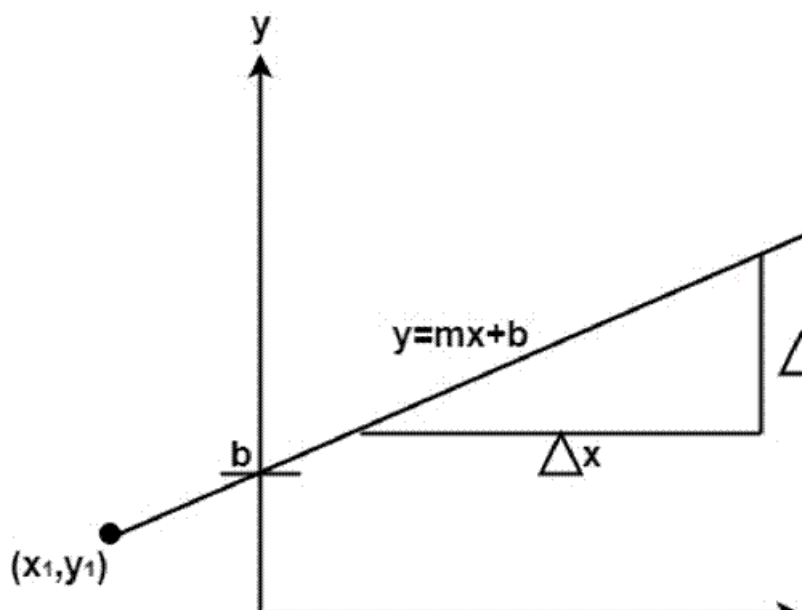
- Points are fundamental for drawing other shapes, as they serve as the building blocks for lines, polygons, and curves.
- The book describes how to represent points in various coordinate systems, like screen coordinates, world coordinates, and camera coordinates.



2.5 Line

A line is another basic graphical primitive that connects two points. In 2D, a line is typically represented by a pair of points, while in 3D, it can be represented by two points or by a parametric equation.

- The authors would explain algorithms for drawing lines on raster displays, with Bresenham's line algorithm being a popular example. This algorithm ensures that the line is drawn most efficiently while minimizing errors when approximating the ideal straight line.
- The book might also cover techniques for antialiasing lines to avoid the "jagged" appearance caused by the discrete nature of pixel grids.

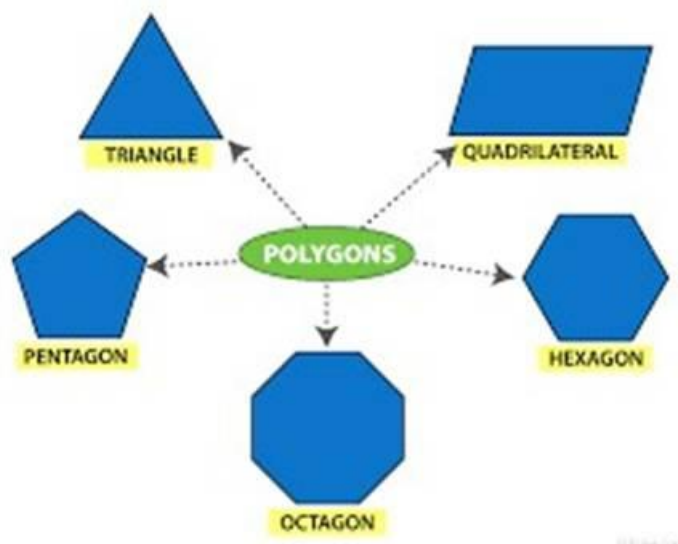


2.6 Polygon

A polygon is a closed shape formed by a sequence of connected line segments. Polygons are fundamental in computer graphics for

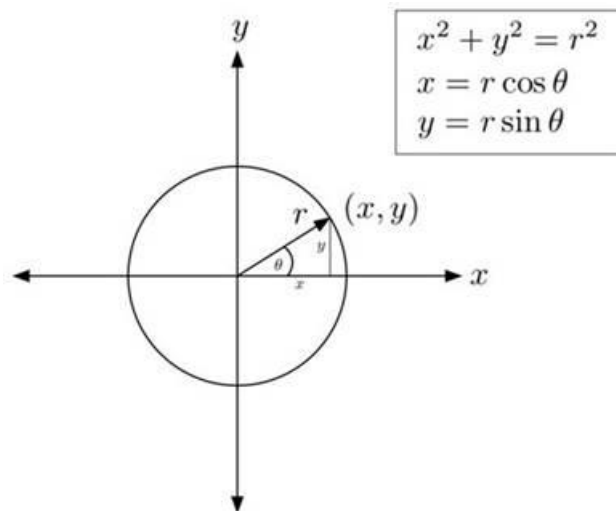
representing objects in a scene. They are often used to approximate more complex shapes, such as 3D models.

- In the book, the authors discuss how polygons can be used to represent both 2D and 3D objects, and how algorithms like scanline fill or ray tracing can be used to render these polygons.
- For 3D graphics, polygons (especially triangles) are often the fundamental unit for rendering, as the book explains how complex models can be broken down into many smaller polygons for easier computation.



2.7 Circle

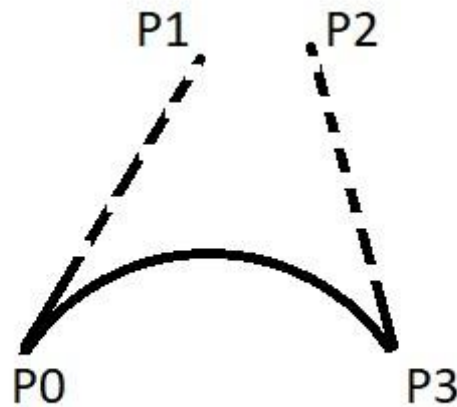
A circle is defined by all points at a constant distance from a center point in 2D space. Circles are important in computer graphics for rendering smooth curves and as part of more complex shapes.



2.8 Curves and Text

Curves and text are other essential primitives in computer graphics.

- Curves can be represented using various mathematical equations, such as Bézier curves or B-splines. The book would explain these concepts in the context of how to represent and render smooth curves for more complex shapes and models.
- For text, the book would describe the representation of characters as shapes (using a font rasterization technique) and the algorithmic approaches used to display text on a screen. One common method discussed might be bitmap fonts, where each character is stored as a grid of pixels, or vector fonts, where characters are represented using curves and lines.



3. Coordinate Conventions

3.1 Scan conversion a Line

A line connects two points and serves as a fundamental element in graphics. To draw a line, you need two points between which the line is formed. In geometry, a line or line segment can be uniquely defined by these two points.

Algebraically, a line can also be represented by its **slope** (denoted by m) and **y-intercept** (denoted by b).

In computer graphics, a line is typically defined by its two endpoints. However, most line-drawing algorithms calculate the slope and y-intercept as intermediate outputs for rendering purposes.

Line Drawing Algorithms

Due to the limitations of raster displays, line-drawing algorithms aim to create the best approximation of an ideal line. Before delving into specific algorithms, it's important to consider the general requirements for such methods:

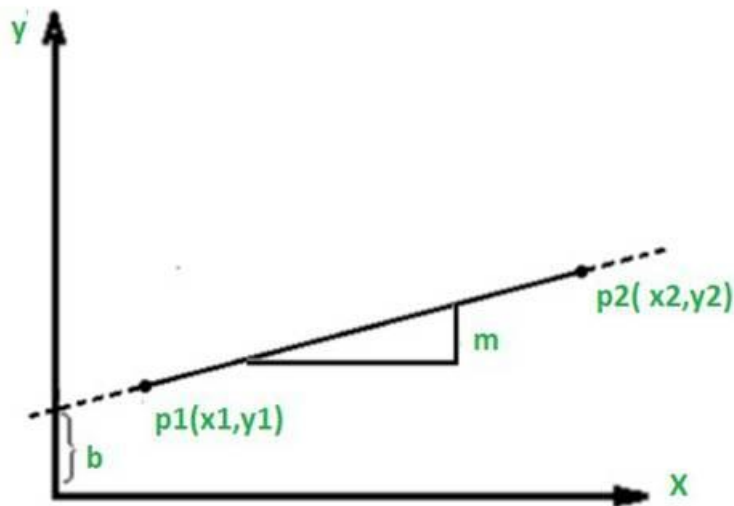
Design Criteria for Line Drawing:

1. **Straightness:** Lines should appear straight and continuous.
2. **Accuracy:** The line should start and end at the correct positions.
3. **Uniform Brightness:** Displayed lines must maintain consistent brightness along their length, regardless of orientation or length.
4. **Speed:** The algorithm should render the line quickly and efficiently.

Method-1: Direct Method

The **Direct Method** for drawing lines involves the following steps:

1. **Input:** Start with two endpoints of the line, say (x_1, y_1) and (x_2, y_2) .
2. **Find the Slope:** Calculate the slope m of the line using the formula: $m = y_2 - y_1$
3. **Line Equation:** Use the slope-intercept form of the line equation: $y = mx + b$
4. **Calculate b:** Substitute one endpoint into the equation to find the y-intercept b .
5. **Generate Points:** Increment x step by step, calculate the corresponding y value for each x , and determine the intermediate points.
6. **Plot Points:** Draw the line by plotting all calculated points on the raster display.



Here's a refined version of the explanation for the **DDA (Digital Differential Analyzer) Algorithm**:

Method-2: DDA (Digital Differential Analyzer) Algorithm

The **DDA Algorithm** is an incremental technique used for rasterizing a line between two given endpoints. It computes intermediate points between the start and end points of a line, ensuring that the line appears smooth on a pixel grid.

Basic Idea:

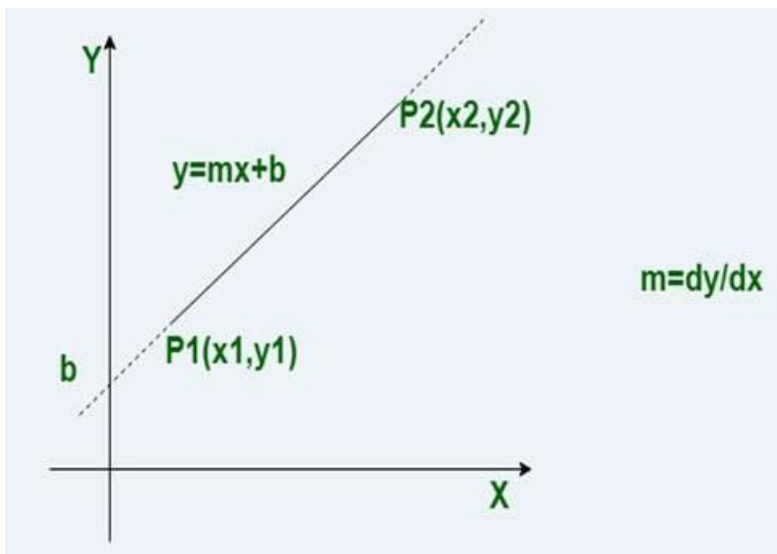
- The algorithm calculates the slope m of the line using the difference in x -coordinates (Δx) and y -coordinates (Δy) of the endpoints:
- Depending on the slope, the algorithm increments one coordinate (either x or y) at fixed steps while calculating the other coordinate using the slope equation.

Algorithm Steps:

1. Start with the initial point $(x1, y1)$.
2. Incrementally calculate the next pixel coordinates based on the value of the slope m :
 -
3. Repeat this process until the endpoint $(x2, y2)$ is reached.

Key Properties:

- The algorithm works for all types of lines, including horizontal, vertical, and diagonal.
- The increment of coordinates depends on the magnitude of the slope:
 - **Positive slope:** Coordinates increase.
 - **Negative slope:** Coordinates decrease.



Here's a refined explanation of **Bresenham's Line Generation Algorithm**:

Method-3: Bresenham's Line Generation Algorithm

The **Bresenham's Algorithm** is an efficient incremental scan conversion method for drawing lines. It is preferred because it uses only integer calculations, making it faster and more suitable for computer graphics.

Key Idea:

- The algorithm calculates points along the line path by evaluating a **decision parameter** to determine the nearest pixel.

- It minimizes floating-point arithmetic by using integer operations, ensuring quick calculations and precise pixel placements.

Steps of the Algorithm:

Let the endpoints of the line be (x_1, y_1) and (x_2, y_2) . Define:

$$\Delta x = x_2 - x_1, \Delta y = y_2 - y_1$$

Initialize the Decision Parameter:

The initial decision parameter is given as:

Iterate Over the Points:

Starting at (x_1, y_1) calculate the next point based on the decision parameter d :

1. Repeat this process until the endpoint (x_2, y_2) is reached.

Advantages:

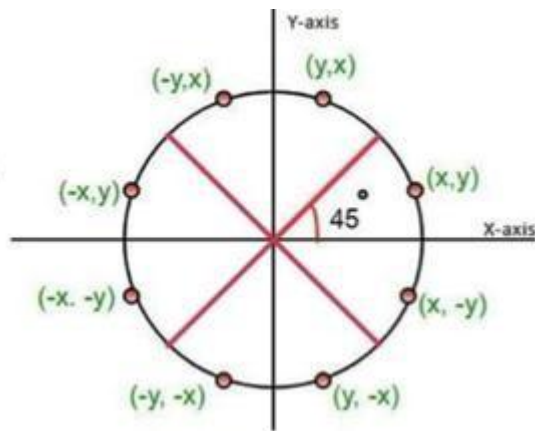
- The algorithm uses only integer addition, subtraction, and bit shifting.
- It is faster than methods requiring floating-point calculations.
- The results are highly accurate, even for lines with steep slopes or small lengths.

3.2 Scan conversion a Circle

A **circle** is defined as a set of points that are all the same distance from a common point. This common point is known as the **center** of the circle, and the **radius** is the distance from the center to any point on its circumference.

A circle has **eight-fold symmetry**, which means that it can be divided into four quadrants. By knowing the position of just one point on the circumference, the circle can be constructed by reflecting this point across the 45-degree axes of symmetry. This symmetry

simplifies the task of drawing a circle on a computer, as only one quadrant needs to be calculated, and the remaining points can be reflected.



Methods of Circle Scan Conversion

Method I – Direct or Polynomial Method

In this method, a circle is defined using a **polynomial equation**. This approach involves representing the circle mathematically by its general equation in Cartesian coordinates:

$$x^2 + y^2 = r^2$$

Where (x, y) are the coordinates of points on the circumference, and r is the radius of the circle. This equation can be used to calculate points on the circle by solving for y given x values, or vice versa.

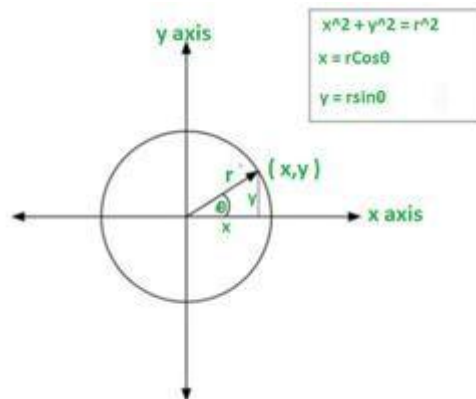
Method II – Polar Coordinates Method

In this method, the coordinates of the points on the circle are converted into **polar coordinates**. The polar form of the circle is expressed as:

$$r = \sqrt{x^2 + y^2}$$

Where r is the radius, and the angle θ helps to determine the position of the points on the circumference. The points are

calculated by iterating over the angles, converting them back into Cartesian coordinates to plot the circle



Circle Generation Algorithms

1) Midpoint Circle Algorithm

The **Midpoint Circle Drawing Algorithm** is used to calculate all the perimeter points of a circle. In this algorithm, the midpoint between two pixels is calculated, which helps in determining the **decision parameter**. This decision parameter decides which pixel should be chosen next to draw the circle. The algorithm calculates the points for one octant and generates the points for the other octants using the **eight-way symmetry** of the circle.

Algorithm:

1. **Plot the initial point:** Set $x=0$ and $y=r$
2. **Calculate the initial decision parameter:** $P=5/4 - r$
3. **Update the decision parameter based on its value:**

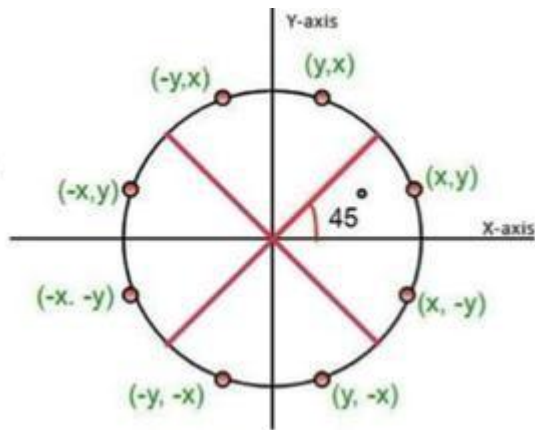
- If $P < 0$:

$$P = P + 2x + 3, \quad xN = x + 1, \quad yN = y$$

- If $P \geq 0$:

$$P = P + 2(x - y) + 5, \quad xN = x + 1, \quad yN = y - 1$$

4. **Plot the complete circle using 8-way symmetry.**



2) Bresenham's Circle Drawing Algorithm

The **Bresenham's Circle Drawing Algorithm** is an incremental method that calculates the points closest to the circle's boundary. It uses **integer arithmetic**, making the algorithm faster and simpler. The algorithm works by selecting the pixel with the least distance from the true circle boundary and then calculates successive points based on this selection. The algorithm takes advantage of the **eight-fold symmetry** of the circle, so the points calculated for the first octant can be reflected to generate points for the remaining octants.

Algorithm:

1. **Initialize:** set $x=0$ and $y=r$
2. **Calculate the initial decision parameter:** $D = 3 - 2r$
3. **Update the decision parameter based on its value:**

- If $D < 0$:

$$D = D + 4x + 6, \quad xN = x + 1, \quad yN = y$$

- If $D \geq 0$:

$$D = D + 4(x - y) + 10, \quad xN = x + 1, \quad yN = y - 1$$

4. **Plot the complete circle** using **8-way symmetry**.

3.3 Scan conversion of Ellipse

Scan-conversion of an ellipse involves determining which pixels in a raster grid should be turned on to approximate the shape of an ellipse. The goal is to convert the mathematical description of an ellipse (usually given by an equation) into a set of pixel positions that can be displayed on a screen or printed.

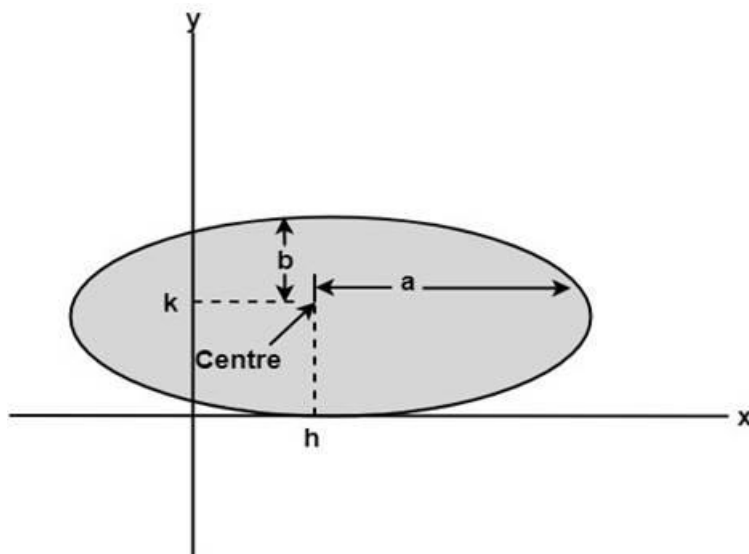
Equation of an Ellipse

An ellipse with its center at (h,k) semi-major axis a (horizontal axis), and semi-minor axis b (vertical axis) has the equation:

$$\frac{(x - h)^2}{a^2} + \frac{(y - k)^2}{b^2} = 1$$

For simplicity, assume the center is at the origin (0,0) so the equation simplifies to:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$



Steps for Scan-Conversion:

1. Region Division (Midpoint Algorithm):

- The ellipse can be divided into two parts based on the ratio of its axes:

■ **Region 1:** When $a \geq b$, the ellipse is wider than it is tall.

■ **Region 2:** When $a < b$ the ellipse is taller than it is wide.

- The scan-conversion process is performed differently for each region.

2. Midpoint Ellipse Algorithm:

- This algorithm is commonly used for ellipse drawing and involves calculating decisions based on the ellipse's equation and its tangent to pixel positions.

3. Region 1: Drawing the Upper Half (for wide ellipses)

- The algorithm begins by plotting the first point (x_0, y_0) on the ellipse, starting from the center.
- The equation of the ellipse helps to define the boundaries and whether the next pixel should be on the curve. For an ellipse, we use:

$$\Delta E_1 = b^2(2x + 1) \quad \text{and} \quad \Delta E_2 = a^2(2y - 1)$$

These values help determine the next pixel location using decision variables based on error terms, updating as you go along the curve.

4. Region 2: Drawing the Lower Half (for tall ellipses)

- For Region 2, the process is similar but adjusted for the different dimensions of the ellipse.

5. Symmetry:

- Ellipses have fourfold symmetry. After plotting points in one quadrant, the points in the other three quadrants can be reflected across the axes to complete the ellipse.

6. Plotting Points:

- The decision criteria in the algorithm help you determine whether to move horizontally, vertically, or diagonally to the next pixel in the raster grid. Based on these criteria, you plot a point and reflect it across the four quadrants.

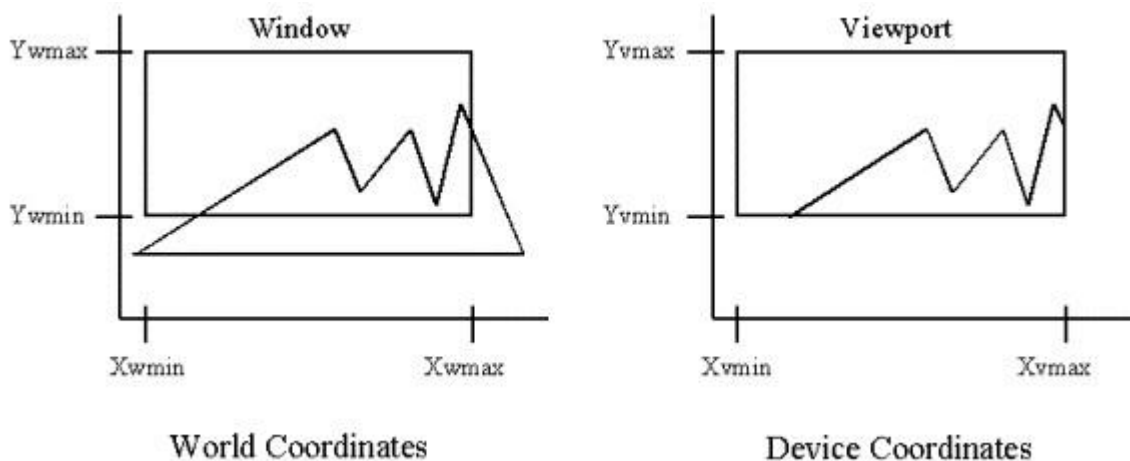
7. Optimizing for Raster:

- Often, the ellipse needs to be approximated more closely based on the pixel grid. To reduce computation and improve accuracy, various

optimizations such as precomputed lookup tables for the decision variables can be used.

3.4 World coordinates

World coordinates are used to represent the position of objects in a scene or environment, typically in 3D space. World coordinates are part of a hierarchical system that allows objects to be positioned and manipulated relative to a defined "world" or global reference system. The concept of world coordinates is essential when transforming objects from one space (such as local object space or camera space) to another (like screen space, which is mapped to pixel coordinates on a display).



World Coordinates Explained:

1. Coordinate Systems:

- **World Coordinate System:** This is a global reference system, often denoted as X_w, Y_w, Z_w for 3D space. All objects in the scene have their positions described in relation to this global reference system.
- **Object Coordinates:** The coordinates that describe an object's position relative to its own local coordinate system. These coordinates are usually transformed into world coordinates through a transformation matrix.
- **Camera/View Coordinates:** The coordinates transformed relative to the camera or view, typically

using a **view transformation**. This helps position the camera and determine what part of the world is visible.

- **Screen/Viewport Coordinates:** The final transformed coordinates that are mapped to pixels on the display screen.

2. How World Coordinates Work:

- Each object in the scene has a position, rotation, and scale in world coordinates, which are typically represented as a point in 3D space (X_w, Y_w, Z_w).
- When an object is placed in the scene, its local coordinate system is transformed into world coordinates via **transformation matrices** (translation, rotation, and scaling matrices).
- These transformations combine to define the position and orientation of an object in the global world.

Coordinate Transformation Process:

To convert object coordinates to world coordinates, you generally use a **transformation matrix**:

- **Translation:** Moving an object to a new position in world space.
- **Rotation:** Rotating an object around an axis in world space.
- **Scaling:** Changing the size of an object in world space.

The transformation matrix is applied to the object's local coordinates to get the world coordinates. This is typically represented as:

$$\begin{pmatrix} X_w \\ Y_w \\ Z_w \end{pmatrix} = \text{Transformation Matrix} \times \begin{pmatrix} X_o \\ Y_o \\ Z_o \end{pmatrix}$$

Where:

- (X_o, Y_o, Z_o) are the object's coordinates in local space.
- (X_w, Y_w, Z_w) are the object's coordinates in world space.

- The **Transformation Matrix** encodes the translation, rotation, and scaling operations.

Converting World Coordinates to Screen Coordinates:

To display the object on the screen, you need to convert world coordinates into **screen coordinates**. This involves several steps:

1. **World to Camera Transformation:** Convert the world coordinates to the camera's coordinate system.
2. **Projection:** Project the 3D coordinates onto a 2D plane (screen space). This is often done through **perspective projection** or **orthographic projection**.
3. **Viewport Mapping:** Map the 2D projected coordinates into the actual pixel space on the screen.

Steps in the Full Process:

1. **Object Coordinates to World Coordinates:**
 - Apply transformations like translation, rotation, and scaling to an object's local coordinates to get its position in the world space.
2. **World Coordinates to Camera Coordinates:**
 - Apply a **view matrix** to convert the world coordinates into the camera's coordinate system.
3. **Camera Coordinates to Clip Space:**
 - Use a **projection matrix** (perspective or orthographic) to convert the camera coordinates into clip space, a 2D space where everything is mapped for the final step.
4. **Clip Space to Normalized Device Coordinates (NDC):**
 - Normalize the clip space coordinates to a range that can be mapped onto the screen.
5. **NDC to Screen Coordinates:**
 - Map the normalized device coordinates (NDC) to the actual pixel coordinates on the screen based on the viewport size.

3.5 DEVICE COORDINATES

Device coordinates are the final step in the graphics pipeline that involves converting world or object coordinates to a form that can be directly rendered to the display. These coordinates are typically in terms of pixels, and the system usually defines the top-left corner of the screen as the origin (0, 0), with the x-axis increasing to the right and the y-axis increasing downwards.

Key Steps Involved in Device Coordinates Transformation:

1. **World Coordinates to Camera Coordinates (View Transformation):**
 - First, you need to transform world coordinates into the camera's coordinate system using a **view transformation matrix**. This allows you to position the camera and determine which parts of the scene will be visible.
2. **Camera Coordinates to Clip Space (Projection Transformation):**
 - After the camera transformation, the coordinates are projected using a **projection matrix** (perspective or orthographic). This projects the 3D world into a 2D space, essentially flattening the scene onto a plane.
3. **Clip Space to Normalized Device Coordinates (NDC):**
 - Once in clip space, the coordinates are normalized into a unit cube (or a normalized 2D range). This is where the x and y values typically range from -1 to 1, ensuring that the scene fits within the viewport and is properly scaled.
4. **NDC to Device Coordinates:**
 - Finally, the normalized coordinates are mapped to actual device coordinates. This step involves converting the normalized coordinates into pixel values on the screen. For example, if the viewport size is 1920x1080 (width x height), the device coordinates correspond to pixel positions on this grid.

Device Coordinates Representation:

- **2D Device Coordinates:** In 2D rendering, device coordinates correspond to pixel positions on the screen. The x-axis corresponds to horizontal positions, and the

y-axis corresponds to vertical positions. If the screen resolution is $W \times H$, the device coordinates are:

(x_d, y_d) where $0 \leq x_d < W, 0 \leq y_d < H$ where x_d, y_d are the pixel positions on the device.

- **3D Device Coordinates:** In 3D rendering, the device coordinates may involve depth information (z-axis), but typically, this is represented in 2D after the projection step, with a 2D coordinate system on the screen.

Example Transformation Pipeline:

1. World to Camera Transformation:

- Object coordinates are first transformed to the camera's coordinate system using a **view matrix** (for camera positioning).

2. Projection:

- The scene is then projected from the 3D world coordinates to 2D using a **projection matrix**, either perspective (for 3D scenes) or orthographic (for 2D/3D scenes).

3. Viewport Mapping (NDC to Device Coordinates):

- The normalized device coordinates (which are in the range $[-1, 1] \times [-1, 1]$) are mapped to actual pixel coordinates. If the viewport or screen size is $W \times H$, the mapping is usually performed as:

$$x_{device} = \frac{(x_{NDC} + 1)}{2} \times W$$

$$y_{device} = \frac{(y_{NDC} + 1)}{2} \times H$$

○

4. Final Rendering:

- After the transformation, the final device coordinates are used to render the object or image onto the screen.

3.6 Normalized device coordinates

Normalized Device Coordinates (NDC) refer to a standardized coordinate system used in computer graphics to map points to the device's coordinate system. NDCs are independent of the actual device's resolution, aspect ratio, and other hardware-specific factors, making it easier to work with different displays and renderings.

Key Concepts:

1. Range:

- NDCs are typically defined in a normalized space where coordinates range from **-1 to 1** along both the x-axis and the y-axis.
 - For example, in a 2D NDC space, the coordinates (0, 0) represent the center of the screen, while (1, 1) represents the top-right corner, and (-1, -1) represents the bottom-left corner.

2. Device Independence:

- NDCs provide a device-independent way of specifying points and geometry. The actual mapping from NDC to the device's pixel coordinates is done later through a **view transformation**.

3. Transformation:

- The transformation from world coordinates to NDC involves scaling and translating the coordinate system. This allows the same geometric information to be used across different devices and resolutions.
- For instance, you can define the viewport size, and then the NDC system can be mapped to the screen pixels accordingly.

Steps to convert to NDC:

1. **World Coordinates** are first mapped to a **viewing space**.
2. The **viewing coordinates** are then scaled to fit into the **NDC** range (usually -1 to 1 for both axes).
3. The **NDC** coordinates are mapped to **device coordinates** (screen coordinates).

Example:

- Consider a viewport with a range of $0 \leq x \leq 8000$ and $0 \leq y \leq 6000$ (representing the resolution of a screen).
- You would convert the coordinates of a point in world space (e.g., (400,300)) into the range of $-1 \leq x \leq 1$ and $-1 \leq y \leq 1$ to achieve NDC.

3.7 Coordinate Reference Frames (CRFs)

A **Coordinate Reference Frame (CRF)** is a system used to define the position and orientation of points, lines, or objects in space. It provides a consistent way to describe where something is located in a specific coordinate system, making it easier to transform and interpret spatial data across different systems. CRFs are fundamental in fields such as computer graphics, geographic information systems (GIS), navigation, and robotics.

Key Concepts:

1. Origin and Axes:

- A CRF consists of an **origin** (a fixed reference point) and **axes** (usually perpendicular to each other in 2D or 3D), which define the directions in space.
- The axes are typically aligned with some predefined directions like **x**, **y**, and **z** in 3D space.

2. Global vs. Local CRFs:

- **Global CRF:** A coordinate system that is fixed and used universally, such as the Earth's coordinate system (latitude, longitude, and altitude).
- **Local CRF:** A coordinate system that is defined relative to a specific object, scene, or reference point. For example, the local CRF might be defined for an object within a computer graphics scene.

3. Coordinate Transformation:

- CRFs enable transformations between different coordinate systems. For instance, an object might be described in a local CRF relative to its own reference point, but to integrate it into a larger scene, its

position must be converted to a global CRF (like world coordinates).

- This transformation process is achieved using **translation**, **rotation**, and sometimes **scaling** operations.

4. Use in Graphics and GIS:

- In **Computer Graphics**, CRFs are used to represent the position of objects within a scene, where objects may be located in different CRFs (local CRFs for objects, and global CRFs for the entire scene).
- In **Geographic Information Systems (GIS)**, CRFs help relate geographical data (latitude, longitude, and elevation) to a particular map or region on Earth. For example, **WGS84** is a commonly used global CRF for GPS coordinates.

Example:

- A **global CRF** might have the origin at the center of the Earth, with axes pointing along the Earth's rotation, and be used to map locations in terms of latitude and longitude.
- A **local CRF** could be defined at the center of a robot. The robot might have an origin at its center, and the axes might correspond to the forward direction and the sides of the robot, helping to describe the robot's position in its own frame.

Importance:

1. **Consistency:** CRFs provide a standardized reference for measurements, ensuring consistency when dealing with spatial data.
2. **Flexibility:** Different CRFs can be used depending on the context. For instance, an object's position may be described in the object's local CRF, but for global interaction, its coordinates must be converted to the global CRF.
3. **Transformations:** CRFs are essential for understanding and applying transformations such as rotation and translation in graphics, robotics, and other applications.

Coordinate System Transformations:

- A key use of CRFs is in transforming data from one reference frame to another. For example, to move an object in 3D space, you may need to convert its coordinates from one CRF (local to the object) to another (local to the scene or global world coordinates).

3.8 Zooming by Changing Coordinate Reference Frames (CRFs)

Zooming in computer graphics or spatial data manipulation refers to changing the scale of the view to either get a closer or broader perspective of an object or scene. One effective way of zooming is by changing the **Coordinate Reference Frame (CRF)**, which involves modifying the scale of the coordinates in a specific reference frame without altering the actual data. This can make it easier to manipulate the view of a scene or object from a different perspective, often by transforming its coordinates.

How Zooming Works Using CRFs:

When zooming, the idea is to **scale the coordinates** within a CRF. Changing the CRF essentially alters the **viewport** or **field of view** by scaling the coordinates, and this transformation can give the appearance of zooming in (making objects appear larger) or zooming out (making objects appear smaller).

Zooming In:

- Zooming in can be achieved by **shrinking the coordinate space**. In this case, the coordinates within the CRF are scaled in a way that brings objects closer to the viewer's perspective, making them appear larger on the screen.
- The CRF gets **scaled down** to a smaller region, increasing the apparent size of the objects.
- For example, if the zoom factor is **2x**, the coordinates in the CRF will be scaled such that the object occupies a larger portion of the view.

Zooming Out:

- Zooming out is the opposite; it involves **expanding the coordinate space**. The CRF is scaled up, so objects appear smaller and more of the scene becomes visible.
- The view of the objects expands, and the CRF coordinates are scaled up to show more of the scene or object in the same space.
- For example, if the zoom factor is **0.5x**, the coordinates in the CRF will be scaled such that the object appears smaller.

Mathematics of Zooming:

Zooming by changing CRFs typically involves **scaling the coordinates** according to a zoom factor (zoom coefficient). Let's consider a 2D example:

- Let (x,y) be a point in the **original CRF**.
- When zooming by a factor z , the new coordinates (x',y') in the zoomed CRF are calculated as: $x'=z \cdot x$ $y'=z \cdot y$
Where:
 - $z>1$ for zooming in (making objects larger),
 - $z<1$ for zooming out (making objects smaller).

Zooming In and Out in 3D:

For 3D graphics or spatial data, zooming involves scaling the coordinates along the **x**, **y**, and **z** axes. The transformation becomes:

Where the zoom factor z scales the coordinates on all axes, effectively zooming in or out in 3D space.

Impact of Changing CRFs on the View:

1. Viewport Adjustment:

- Zooming by changing CRFs modifies the region visible in the viewport. By scaling the CRF, we can adjust the area that fits into the display.

2. World vs. View Coordinates:

- The zooming transformation occurs in the **view coordinates**, but it can affect how the object appears

in the **world coordinates** when mapped back to the screen. Thus, zooming adjusts the way objects are seen from a particular viewpoint.

Applications of Zooming by Changing CRFs:

1. 2D and 3D Graphics:

- Zooming is commonly used in both 2D and 3D graphics to control how much of the scene is visible and how large the objects appear.
- In games and simulations, the camera view can be zoomed in or out by altering the CRF.

2. Geographical Information Systems (GIS):

- In GIS applications, zooming is used to change the scale of a map or satellite image by adjusting the CRF, allowing users to zoom into a specific area or zoom out to view a larger region.

3. Medical Imaging and CAD Software:

- In medical imaging (like MRI or CT scans) and computer-aided design (CAD) software, zooming is essential for focusing on specific parts of a model or image, helping professionals to view details more clearly.

3.9 Panning by Changing Coordinate Reference Frames (CRFs)

Panning in computer graphics refers to the process of moving the view of a scene horizontally or vertically (or in 2D/3D space) without changing the zoom level. It gives the effect of shifting the perspective of the scene from one point to another, often used to explore different parts of a scene, map, or model.

When **panning** is done by changing the **Coordinate Reference Frame (CRF)**, the coordinate system is shifted in space, effectively shifting the view of the objects in the scene. The scene itself doesn't change, but the **camera or viewpoint** moves, which changes the reference frame used to view the objects in the scene.

How Panning Works Using CRFs:

Panning by changing CRFs involves translating the coordinates of the objects in a scene to shift the viewpoint. By applying a translation transformation, the entire view of the scene can be moved along the desired axis (usually the x or y axis in 2D, or x, y, and z in 3D).

Steps in Panning:

1. Coordinate Transformation:

- The primary concept in panning is shifting the **reference frame** to translate the scene.
- The **coordinate system** itself is translated by a fixed amount along the x, y, and/or z axis.
- Mathematically, if an object has coordinates (x,y)(x, y)(x,y) in the original reference frame, panning by Δx and Δy involves translating the coordinates by these amounts:

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

Where:

- x' and y' are the new coordinates after panning.
- Δx and Δy are the pan distances along the x and y axes, respectively.

Panning in 3D: In a 3D coordinate system, panning involves shifting along the x, y, and z axes:

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

$$z' = z + \Delta z$$

1. Where Δz is the shift along the z-axis (usually for 3D scenes).

Impact of Changing CRFs on the View:

- **Translation of the Viewport:**
 - The view is effectively moved without altering the objects or their details.
 - The **objects** appear to move in the opposite direction of the panning motion, giving the user the sense that they are exploring different parts of the scene.
 - Panning allows users to **move around the scene** to explore areas outside the current viewport.
- **Viewport Shift:**
 - The coordinates are adjusted by a certain amount, and this shift represents a movement of the **viewport** or **camera**. The objects themselves are unchanged, but the reference frame is modified to show a different part of the scene.
 - Unlike zooming, which changes the scale of objects, panning only shifts the view of the objects.

Mathematics of Panning:

- For a 2D scene, the transformation involves adding a translation vector Δx and Δy to the coordinates:

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

For a 3D scene, the translation is done similarly in all three axes:

$$x' = x + \Delta x$$

$$x' = x + \Delta x$$

$$y' = y + \Delta y$$

$$z' = z + \Delta z$$

Applications of Panning by Changing CRFs:

1. **2D and 3D Graphics:**
 - In video games, simulations, and graphic design, panning is used to shift the view around a scene,

allowing players or users to explore different parts of the world.

- For instance, panning is common in **map navigation** or **CAD software**, where users can move the viewport across a larger design or area.

2. **Geographical Information Systems (GIS):**

- In GIS applications, panning is used to move around a map. As users pan, the coordinate reference frame changes to show a different region of the map without altering the map's scale or orientation.

3. **Video and Image Editing:**

- In video editing software, panning allows users to shift the view across a scene without changing its zoom level, enabling dynamic camera effects or editing views.
- In image editing, panning can help users move across a large image or artwork.

4. **Camera Controls in 3D Environments:**

- In 3D modeling and animation, panning allows the user to move the camera around the scene without altering the scene's scale, offering a better view of different parts of the model.

REFERENCES

Book:

1. *D. Hearn and M. P. Baker, "Computer Graphics with OpenGL", 4th Ed.,*

Pearson Education, 2013. [Link](#)

2. *E. Angel, "Interactive Computer Graphics: A Top-Down Approach Using*

OpenGL", 5th Ed., Pearson Education, 2009. [Link](#)

JavatPoint: *Computer Graphics Overview :-[Link](#)*

GeeksforGeeks: Computer Graphics :- [Link](#)

Online Resources: Coursera and edX courses on graphics Design.

GNU Tools: Documentation for Lex, Yacc, GCC, and related tools.